Solid programming

Paolo Burgio
paolo.burgio@unimore.it

“Bad programmers worry about the code. Good programmers worry about data structures and their relationships.”

Linus Torvalds

What will we see?

- ~~How to implement patterns within a given language~~
- ~~How to choose variables' names~~
- ~~CamelCase vs snake_case~~
- ~~Best coding practices~~

How to code properly

- "Properly?"
- In such a way that your code becomes scalable, maintainable, robust..
- Do engineer's job!

SOLID programming

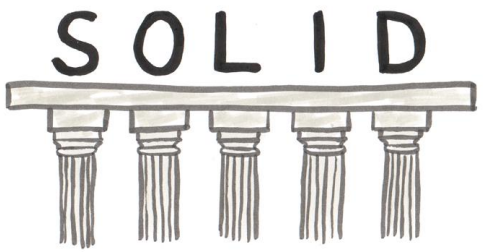
Aka: **Object Oriented Design**

Five principles that ~~save your life~~ make the difference between a programmer/coder and a software architect

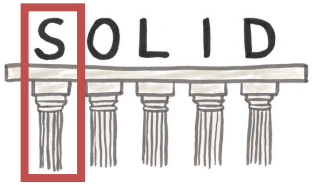
› (Or between an happy person and a sad person)

Applicable to object-oriented programming (but not only)

1. Single Responsibility
2. Open/Close principle
3. Liskov substitution
4. Interface segregation
5. Dependency inversion



Single responsibility principle

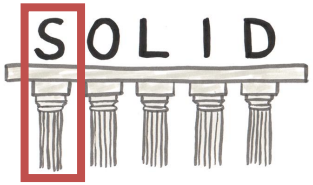


A software entity should have only one reason to change

- Aka: every class should have a single responsibility or single job or single purpose

- **Answers to**: "What should I put into a class?"
- **Pros**: you always know where/what/how to change your code, and don't mess up things
- (**Cons**: increase number of classes and effort…)

Single responsibility principle



Streamline from *divide et impera* design phase

We use it for classes, can also be applied to any kind of SW entity

› Classes, Modules, Interfaces, …

As simple as this: add a comment to your class/entity

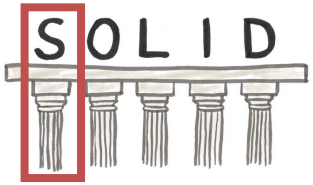
1. Company of rubber duck is recommended
2. Code smell warning: if you cannot write a meaningful and short comment, then this class has too many responsibilities! ("Bloater") – back to design phase

Project: **WebSvc.java**

```
/**
 * Interface to persistence layer
 */
public interface IDb {
  // ...
}
```

```
/**
 * IDb Implementation with RAM data structures.
 * For local testing. Does not persist.
 */
public class InMemoryDb implements IDb {
  // ...
}
```

Single responsibility principle



This class does too many things!

- On one side, it is an entity (Aka: Data Object)
- On the other side, it implements a business functionality (*calcArea*)

Project: OfRectanglesAndSquaresNoSolid.Java

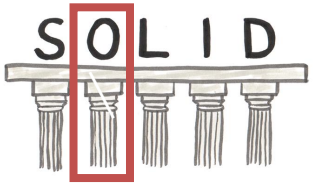
```
public class Rectangle {
  private double _height;
  private double _width;

  public void setHeight(double height) {
    this._height = height;
  }

  // ...

  public double calcArea() {
    return this._height * this._width;
  }
}
```

Refactor the code to make it SOLID

- Hint: the DO and the business functionality are distinct concepts / responsibilities
- Solution is in the project *OfRectanglesAndSquares.Java*

Let's code!

Open/Close principle

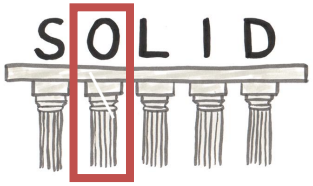


Entities should be open for extension, but closed for modification

- Aka: you should never change anything, always adding new behavior using polymorphism

- **Answers to**: "How should I extend my code?"
- **Pros**: reduce the number of bugs and headaches
- (**Cons**: N/A)

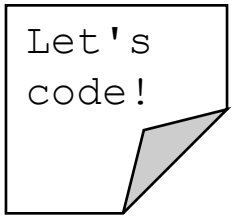
Open/Close principle



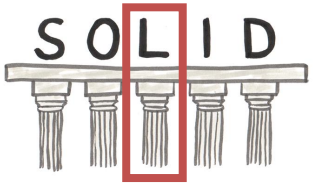
Now, add this functionality to the *Rectangle* class

ID	Description	Parents	Siblings
RECT_11	It shall also be possible to round the width and height of a rectangle while invoking setters. This complements the standard setter where no rounding is performed	…	…

- Remember, you **must** subclass it to adhere to SOLID
- Solution is in the project *OfRectanglesAndSquares.Java*



Liskov substitution principle

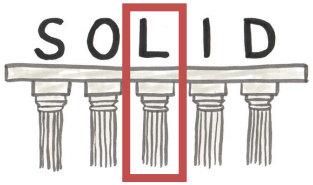


You should be able to substitute any parent class with any of their children without any behavior modification

- Aka: remember "Rectangles vs Squares"

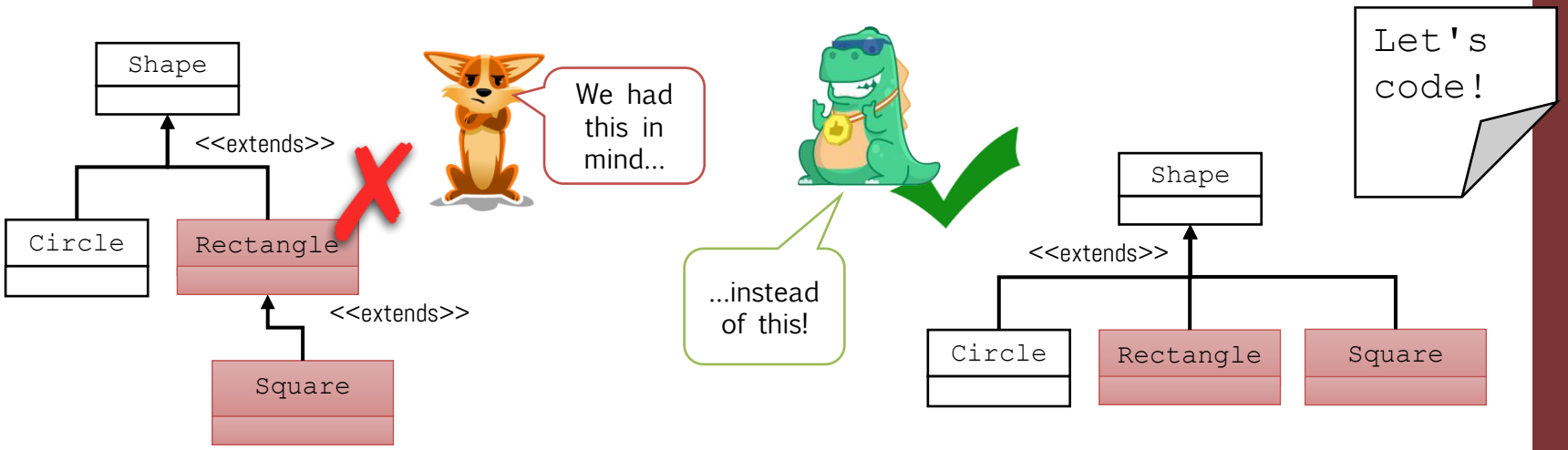
- **Answers to**: "When and what should I inherit?"
- **Pros**: code is scalable, and minimize changes upon modifications
- (**Cons**: you have to think before you code)
  - not actually a con…

Liskov substitution principle



Example: run the project *OfRectanglesAndSquaresNoSolid.Java*

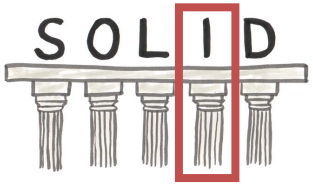
- The main class (*App*) performs functional tests on the *Square* class to check if area doubles by doubling the side
- ...but it fails since we decided that a *Square* is also a *Rectangle*, which is not true in our domain!



Now, refactor it

- It might take time...which **is the time you didn't spend to do a good design phase**, multiplied by a delta that depends on you (mine is typically ~x3)
- Solution is in the project *OfRectanglesAndSquares.Java*

Interface segregation principle



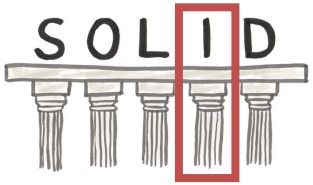
Many client specific interfaces are better than a big one

- Aka: Interfaces should be the minimal set of behaviors you need

- **Answers to**: "When should I create an interface?"
  - Spolier: "as much as you can"
- **Pros**: you minimize dependencies in your code (thus, programming effort)
- (**Cons**: trust me…NONE)

Always remember: an interface is a contract. You can build (automatic) tests over contracts!

Interface segregation principle



This forces us to clearly define and refine contracts/protocols

- Java/C++/C# Interfaces (or header files in C) come directly from this
- Example: *OfRectanglesAndSquares.Java*

It has knowledge of all concrete classes (someone has to!)

For simplicity I omit the <<uses>> relation to *Rectangles* and *Squares*

App
+ main()

IRectangle
+ getWidth()
+ getHeight()

ISquare
+ getSide()

ICalculator
+ calcArea(IRectangle r)
+ calcArea(ISquare r)

<<uses>>

Rectangle
+ getWidth()
+ getHeight()

Square
+ getSide()

<<uses>>

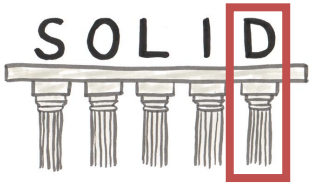
Calculator
+ calcArea(IRectangle r)
+ calcArea(ISquare r)

RectangleWithRound
+ getWidth()
+ getHeight()

CalculatorWithOptimization
+ calcArea(IRectangle r

This has an "optimized" version of *calcArea* for *Rectangles*

Dependency inversion principle

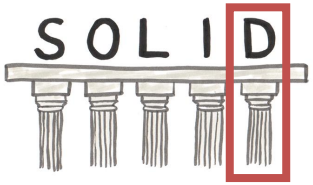


Your project shouldn't depend of anything, make those things depend of interfaces

- Design wrappers around your dependencies
  - (This is **NOT** "dependency injection"…but its good friend)

- **Answers to**: "How can I avoid getting crazy with dependencies?"
- **Pros**: isolation between code components; your code reflects the analysis/model of business
- (**Cons**: additional programming effort)

Dependency inversion principle

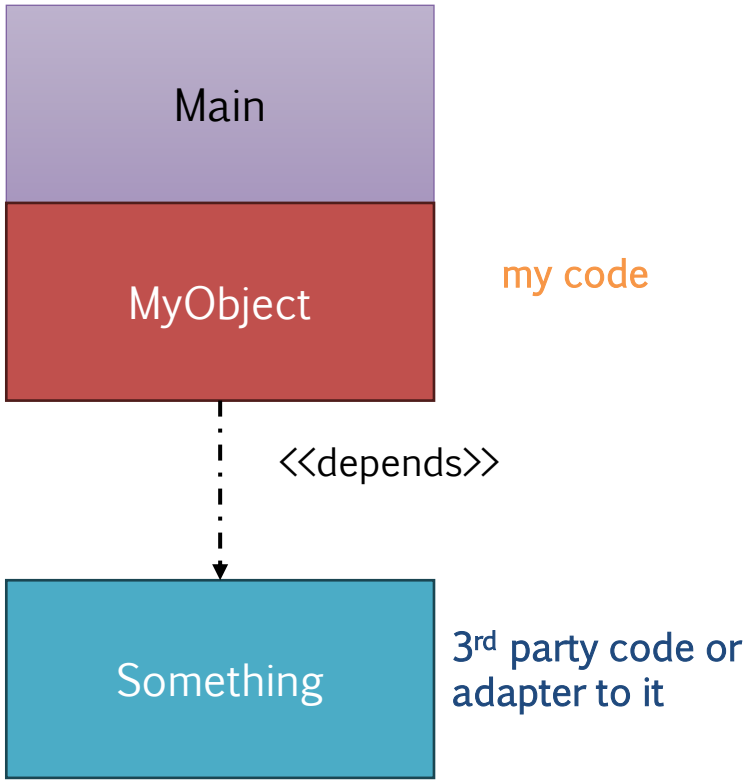


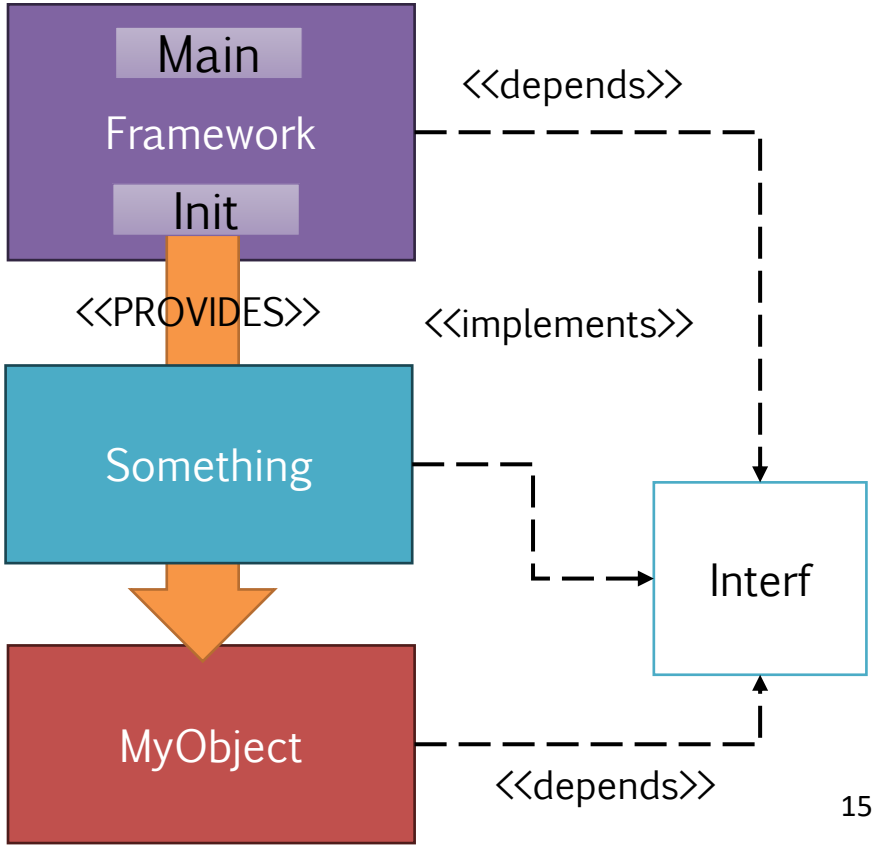
Toolkit/library approach

- Tied to specific drivers/technologies
- Often, tied to 3rd party code

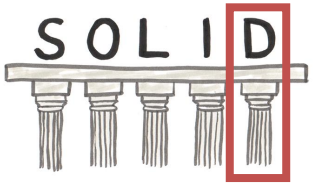
Framework approach

- Inversion of control
- Dependency injection
- Aka: "Service injection"





Example: `WebSvc.Java`

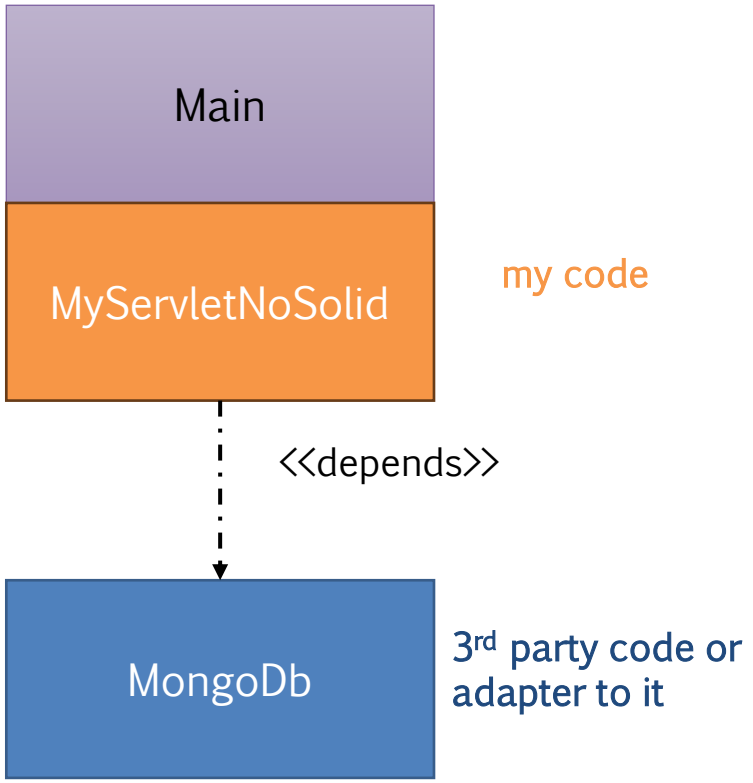


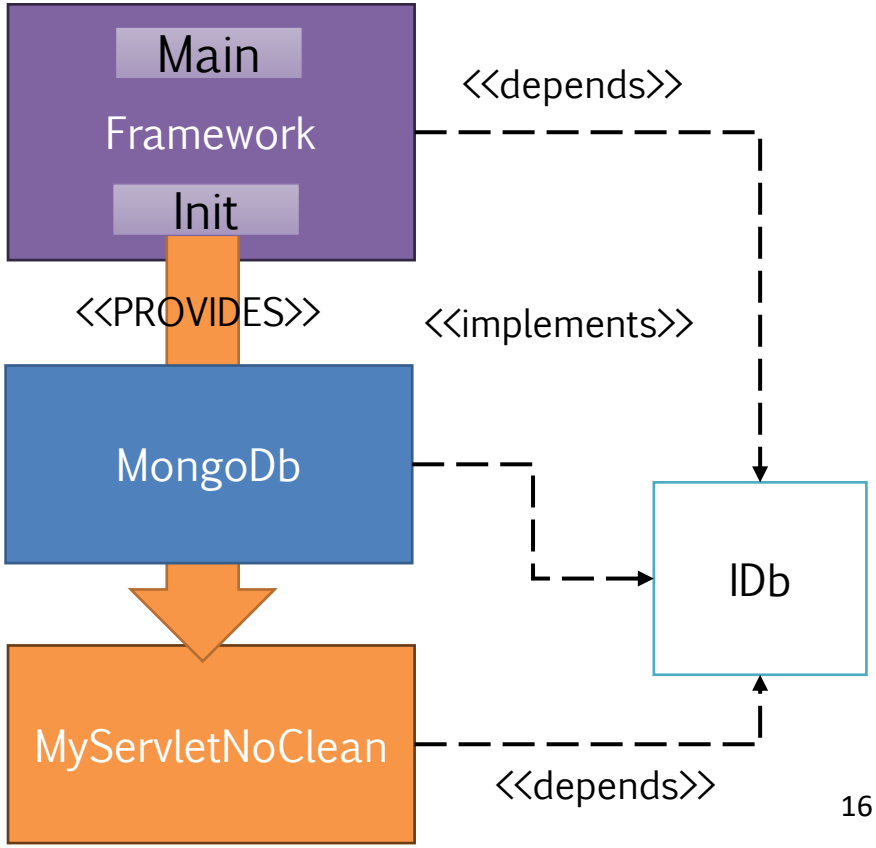
Toolkit/library approach

- Tied to specific drivers/technologies
- Often, tied to 3rd party code

Framework approach

- Inversion of control
- Dependency injection
- Everything is a "Service"





`WebSvc.Java` - three environments

Mock

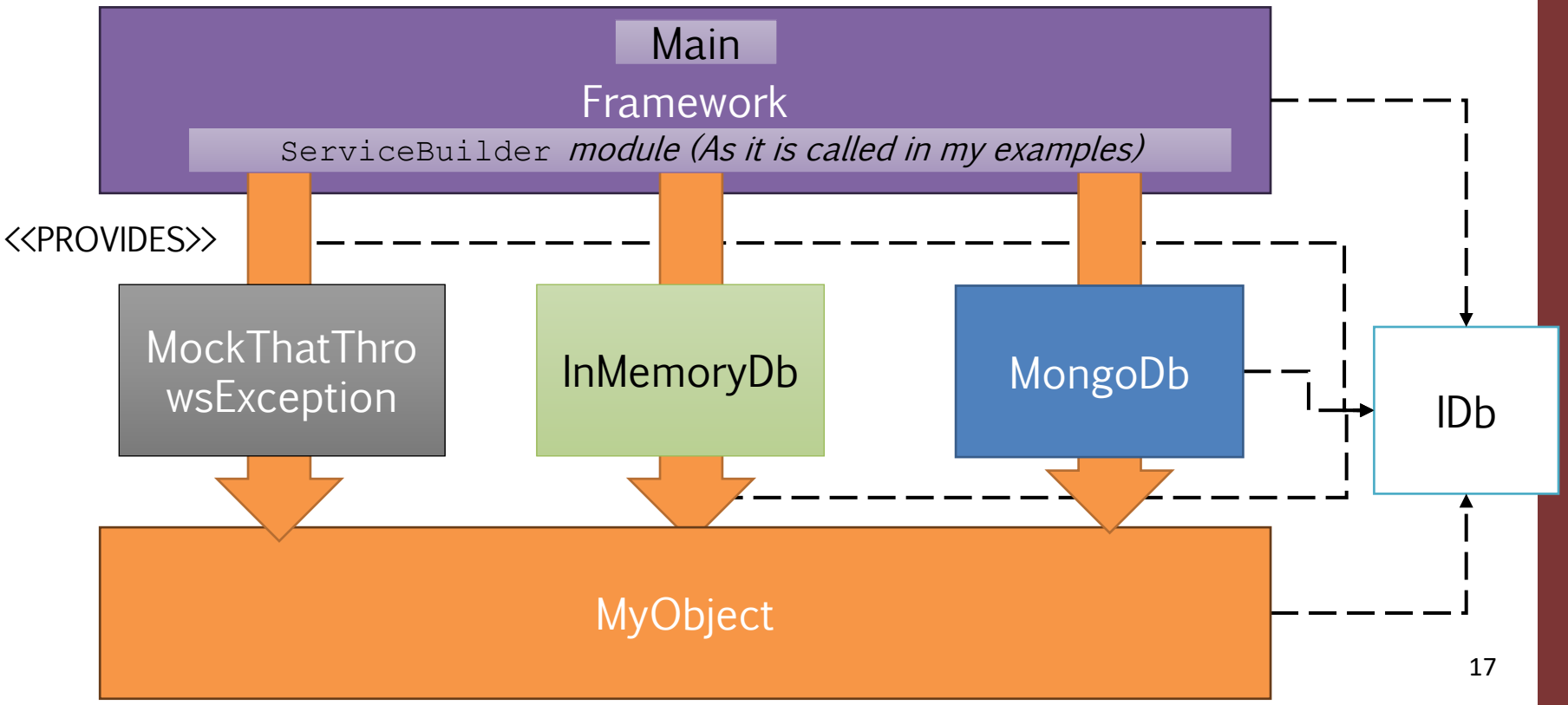
- For testing environment
  - Unit testing
- E.g., always throws exceptions

Local

- Dev environment
- Integration tests
- No MongoDb here

Production

- Complete version
- All dependencies are resolved



References

Course website

- http://hipert.unimore.it/people/paolob/pub/Industrial_Informatics/index.html

My contacts

- paolo.burgio@unimore.it
- http://hipert.mat.unimore.it/people/paolob/

Resources

- https://springframework.guru/solid-principles-object-oriented-programming/
- A "small blog"
  - http://www.google.com